

COMPSCI 701:
Creating Maintainable Software
Lecture 7.1: Alterability and Design Patterns

Ewan Tempero, Yu-Cheng Tu
School of Computer Science

Agenda

- Agenda
- Assignment 3
- Alterability
- Change Ratio
- Design Patterns
- Composite
- Composite and Alterability
- Design Patterns
- Example
- Key Points

- Assignment 3
- Design Patterns
- Composite Pattern

Assignment 3

- Agenda
- **Assignment 3**
- Alterability
- Change Ratio
- Design Patterns
- Composite
- Composite and Alterability
- Design Patterns
- Example
- Key Points

- Add the “new game” and “save/load game” features to Kalah using the Command and Momento Patterns.
- For Command:
 - You must have exactly one “command” interface
 - You must have at least 2 (TWO) concrete commands
 - You must not have the “execute” method of your command called in **requested** (either directly or indirectly) in more than one location in your code
- Briefly explain how you have used the patterns (use template provided)
- Ensure your implementation follows the advice discussed in class regarding maintainable implementations, or justify any (very small number) of departures
- Due Friday 3 October 23:59hrs (End of Week 9)
- More details on Canvas

- Agenda
- Assignment 3
- **Alterability**
- Change Ratio
- Design Patterns
- Composite
- Composite and Alterability
- Design Patterns
- Example
- Key Points

Intuition for Alterability

- Generally, the less we have to change **existing code**, the more alterable the design is:
 - less code to change means less amount of work to do (**efficiency**) and less likely to make a mistake (**effectiveness**)
 - fewer places we need to deal with (switching — (**efficiency**))
 - fewer opportunities to make a mistake (**effectiveness**)

- Agenda
- Assignment 3
- **Alterability**
- Change Ratio
- Design Patterns
- Composite
- Composite and Alterability
- Design Patterns
- Example
- Key Points

Assessing Alterability cont.

- We assess the alterability of a design—if the design does not change, its alterability should not change
- So alterability should be independent of the change case we want to implement—if one change case is “larger” (higher impact) than another, that does not change the alterability of the existing design
 1. **Noughts and Crosses — whether changing the game to use a graphical user interface, or just change 'X' to 'x' and 'O' to 'o', the alterability of the existing design must be the same**
- In practical terms, this means alterability should not depend on the amount of new code that has to be written
- But what is new code?
 - adding code to existing line? **✗**— changing existing line of code
 - adding complete line of code to existing method? **✗**— changing existing method
 - adding new method? **✗**— changing existing class
 - adding new nested class? **✗**— changing existing class
 - adding enum? **✗**— (if nested) changing existing class
 - adding enum? **✓**(if new file)
 - new interface? **✓**(if in a new file)
 - new class? **✓**(if in a new file)
- **⇒ new files should be ignored when assessing alterability**

Model for Alterability

- Agenda
- Assignment 3
- Alterability
- Change Ratio
- Design Patterns
- Composite
- Composite and Alterability
- Design Patterns
- Example
- Key Points

- There isn't a model for alterability so we will create our own — Change Ratio ($CR = \frac{C}{N}$)
- For a given change case with impact I ...
 - Determine the number of “classes” (files) of the existing design (N)
 - Determine the number of those classes (files) that need to be changed while implementing the change case (C)
 - Compare $CR = \frac{C}{N}$ to I
 - If $\frac{C}{N} \approx I$, then the “cost” of implementing this change case is about what is expected
 - If $\frac{C}{N} > I$, then the “cost” of implementing this change case is higher than what we expect (so lower alterability)
 - If $\frac{C}{N} < I$, then the “cost” of implementing this change case is lower than what we expect (so higher alterability)
 - Do the same for every (non-trivial, reasonably likely) change case and “summarise” the set
 - Difficult to do in practice, but does provide a useful heuristic
 - (Fairly coarse-grained view as does not take methods or statements changed into account)

- Agenda
- Assignment 3
- Alterability
- **Change Ratio**
- Design Patterns
- Composite
- Composite and Alterability
- Design Patterns
- Example
- Key Points

Consequences

- A change case with low impact that requires almost every class to change (high C) is considered low alterability
- We want to reduce $\frac{C}{N}$ as much as possible
- Increasing N will lower $\frac{C}{N}$ —provided C doesn't increase at the same rate
⇒ look for excuses to add classes!
- Reducing C will lower $\frac{C}{N}$
- Notes
 - C does **not** include any new classes created, so put as much of the new code as possible in new classes (files), **not by changing existing classes**—so need to choose a design that will allow this
 - A class is counted as changed whether one expression changes or almost every statement changes—but it is likely hard to reduce C by exploiting this
 - Consistent with Bass, Clements, Kazman definition

- Agenda
- Assignment 3
- Alterability
- **Change Ratio**
- Design Patterns
- Composite
- Composite and Alterability
- Design Patterns
- Example
- Key Points

Example

- Noughts and crosses — change 'X' to 'x' and '0' to 'o'.

Impact: 0.05

- Design A: classes `NoughtsAndCrosses` and `Grid`. $N = 2$

Only `NoughtsAndCrosses` has to to change $C = 1$

- $\mathcal{CR} = \frac{1}{2} = 0.5 \gg 0.05 \Rightarrow$ not alterable (for this change case)
- Design B: classes `NoughtsAndCrosses`, `Grid`, and an additional class `Mark` that specifies what gets printed for the two players. $N = 3$

Only `Mark` has to to change

- $\mathcal{CR} = \frac{1}{3} = 0.33 < 0.5$ so Design B alterability is better than for Design A (for this change case, and still $\gg 0.05$)

- Agenda
- Assignment 3
- Alterability
- **Change Ratio**
- Design Patterns
- Composite
- Composite and Alterability
- Design Patterns
- Example
- Key Points

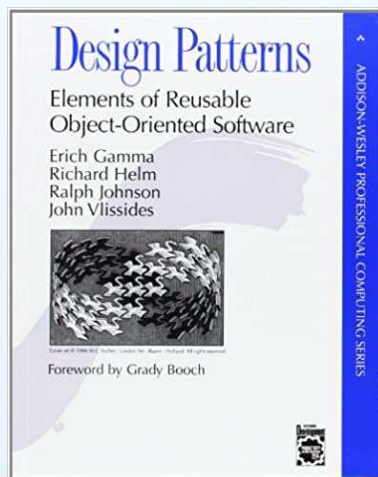
Application

- The lower the CR the “more alterable” the design
- **But!** the CR can only be determined for a given change case. What does it tell us about the overall alterability of a design?
 - One design may have a lower CR than another design for one change case, but a higher CR for a different change case — which is the “more alterable” design?
 - There are an unlimited number of change cases, so how can we be sure we’ve determined the CR for all of them?
 - And when N is in the thousands, how can we be sure we’ve determined C correctly without actually making the change?
- Use of heuristic:
 - When making design decisions, look for designs that **minimise CR** for **likely** and **non-trivial impact** change cases
 - Avoid designs where which design has lower CR highly depends on the change case
 - $\Rightarrow$
 - look for design advice that reduces the number of classes that need to change, or it increases the number of classes while keeping the number of classes that need to change the same
 - for any design, think about change cases that might create problems

Design Patterns

- Agenda
- Assignment 3
- Alterability
- Change Ratio
- Design Patterns
- Composite
- Composite and Alterability
- Design Patterns
- Example
- Key Points

- a mechanism for expressing object-oriented design experience Gamma, Helm, Johnson, Vlissides “Design Patterns: Abstraction and Reuse of Object-Oriented Design” ECOOP 1993
- a **general solution** to a **commonly** occurring problem within a **given context** in software design
 - general solution $\Rightarrow$ not a specific implementation—there can be variations
 - solution $\Rightarrow$ not an alternative (or set of alternatives)
 - common $\Rightarrow$ has been used in multiple systems and so should expect to use it in more
 - context $\Rightarrow$ if the context is different then the design pattern should not be used
- do not, by definition, support Alterability—but as many problems are effectively “how to support change”, design patterns tend to have some impact on Alterability
- Concept is by Gamma, Helm, Johnson, Vlissides, also known as the “Gang of Four”, or GoF



Design Patterns and “Good Design”

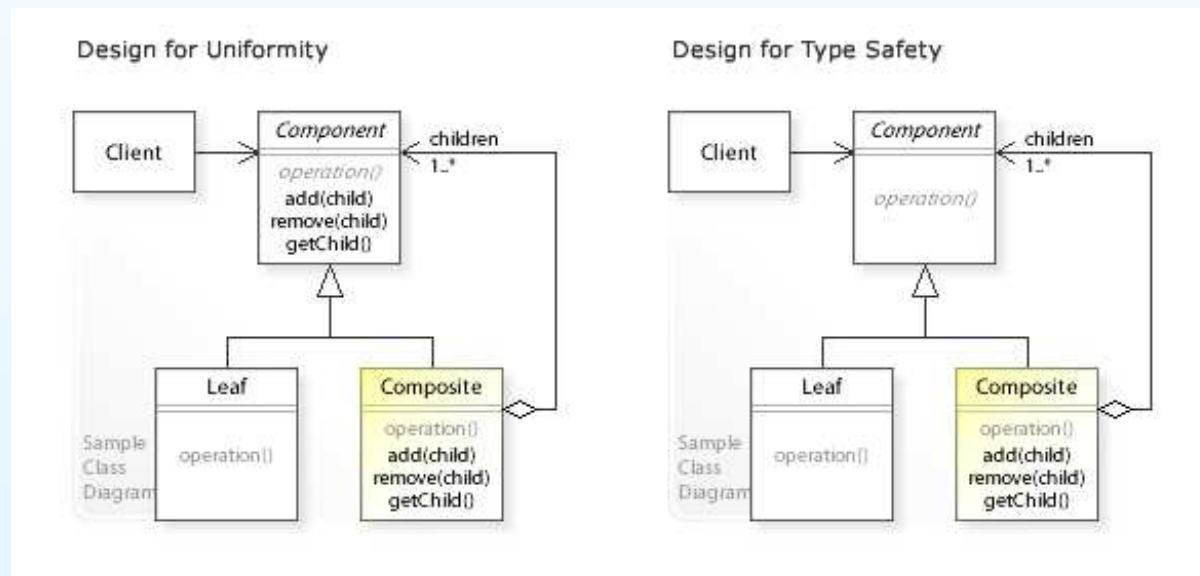
- Agenda
- Assignment 3
- Alterability
- Change Ratio
- Design Patterns
- Composite
- Composite and Alterability
- Design Patterns
- Example
- Key Points

- How can we determine design quality using the concept of “design pattern”? The (wrong) argument:
 - Design patterns are good designs
 - $\Rightarrow$ good designs use design patterns
 - $\Rightarrow$ more design patterns = better design!
- If the problem solved by a particular design pattern is not what needs to be solved, or the context of the problem being solved does not match that of the design pattern, then using that design pattern cannot be considered “good design”

- Agenda
- Assignment 3
- Alterability
- Change Ratio
- Design Patterns
- **Composite**
- Composite and Alterability
- Design Patterns
- Example
- Key Points

Composite

- Intent “Compose objects into tree structures to represent part-whole hierarchies. Composite lets clients treat individual objects and compositions of objects uniformly.”
- **Problem**: represent part-whole hierarchy
- **Context**: need to treat individual parts the same as the whole
- **Solution**: 3 elements
 - **Component** — represents both part and whole, the Client only uses this (in theory)
 - **Leaf** — represents part, can be treated as a Component
 - **Composite** — represents a collection of parts, can be treated as a component
- Not to be confused with **Composition**



(w3sDesign.com)

- Agenda
- Assignment 3
- Alterability
- Change Ratio
- Design Patterns
- **Composite**
- Composite and Alterability
- Design Patterns
- Example
- Key Points

Example uses

- File systems — Directories (Composite) contain Files (Leaves)
- Employee Hierarchies — Departments (Composite) contain Individual Employees (Leaves)
- User interface Frameworks — e.g. Swing
- Arithmetic expressions — values or variables (Leaves), expressions using operators (Composite)

Applying Composite

- Agenda
- Assignment 3
- Alterability
- Change Ratio
- Design Patterns
- **Composite**
- Composite and Alterability
- Design Patterns
- Example
- Key Points

```
public interface File {
    String longDetails();
    String shortDetails();
}

public class PlainFile implements File {
    private String _name;
    public PlainFile(String name) { ... }
    public String longDetails() {
        return _name;
    }
    public String shortDetails() { ... }
}

public class Directory implements File {
    ...
    private List<File> _contents;
    public Directory(String name, String permissions) {
        ...
        _contents = new ArrayList<File>();
    }
    public void addFile(File file) {
        _contents.add(file);
    }
    public String longDetails() {
        String result = _name + "/" + (" + _permissions + ");
        for (File file: _contents) {
            result += "\n" + file.longDetails();
        }
        return result;
    }
    public String shortDetails() { ... }
}
```

- Agenda
- Assignment 3
- Alterability
- Change Ratio
- Design Patterns
- **Composite**
- Composite and Alterability
- Design Patterns
- Example
- Key Points

Using Composite

```
public class Browser {
    public static void main(String[] args) {
        Directory root = new Directory("root", "drwx-----");
        root.addFile(new PlainFile("README"));
        Directory home = new Directory("home", "drwx-----");
        root.addFile(home);
        Directory user = new Directory("ewan", "drwxrwxrwx");
        home.addFile(user);
        PlainFile compsci701quiz2 = new PlainFile("MostSecret.tex");
        user.addFile(compsci701quiz2);
        System.out.println("Printing root directory");
        print(root);
        System.out.println("\nPrinting user directory");
        print(user);
        System.out.println("\nPrinting plain file");
        print(compsci701quiz2);
    }

    private static void print(File start) {
        System.out.println(start.longDetails());
    }
}
```

- Achieving this behaviour without composite would take a lot more code that is a lot more complicate
- Anyone who knows the pattern (it is in their Knowledge Base), will probably find it easier to understand
- (Anyone who doesn't know the pattern may be mystified)
- So probably more **Comprehensible**
- But, **Alterable?**

Composite and Alterability

- Agenda
- Assignment 3
- Alterability
- Change Ratio
- Design Patterns
- Composite
- **Composite and Alterability**
- Design Patterns
- Example
- Key Points

- Remember $\frac{C}{N}$
- To add a new kind of leaf class L
- L is new code, and so “doesn’t count” when evaluating alterability
- One class (at least) will need to change to construct objects from L (and if more than one class needs to change that suggests composite wasn’t really implemented as it should have been)
- So $C = 1$ (all other things being equal, which of course they’re not)
- Adding a new composite is the same
- Typically the client will *not need to change*
- Note: Changing the component may require changing the composite and every leaf!

- Agenda
- Assignment 3
- Alterability
- Change Ratio
- Design Patterns
- Composite
- Composite and Alterability
- Design Patterns
- Example
- Key Points

Points to Note

- There is no interface or class called “Component”
Component is a **role** that some element of the design (in this case by Java interface) plays in the Composite Design Pattern.
- Similarly there is nothing with the name “Composite” or “Leaf”
- $\Rightarrow$ just having classes (or similar) with the name Component, Composite, and Leaf organised according to the design pattern **does not mean** you have used the Composite Design Pattern correctly
- Correct use of a Design Pattern requires using it for solving the relevant problem in the proper context
 - `Browser#print()` shows that `Directory` and `PlainFile` can be treated uniformly

- Agenda
- Assignment 3
- Alterability
- Change Ratio
- Design Patterns
- Composite
- Composite and Alterability
- Design Patterns
- **Example**
- Key Points

Summary

Role	Type
Component	File
Composite	Directory
Leaf	PlainFile
Client	Browser

- Agenda
- Assignment 3
- Alterability
- Change Ratio
- Design Patterns
- Composite
- Composite and Alterability
- Design Patterns
- Example
- Key Points

Key Points

- A possibly useful heuristic for alterability is to look at designs with many classes (increasing N) that mostly do not need to change (decreasing C)
- Design Patterns — captures design experience for solving certain problems in certain contexts
- Some patterns improve alterability